

* Linux환경에서의 C 프로그래밍

- 라이브러리(library) 사용 및 작성 방법
- make 도구 사용 방법(C 프로그램 개발 도구)

GCC

- C 언어의 개요 및 특징

- Unix를 다시 쓰기 위한 언어로 Dennis Ritchie에 의해 설계
- 타 기종간의 이식성(Portability)과 유연성(Flexible)을 지닌 강력한 언어
- 간결(Compact)한 문법구조와 실행속도가 매우 빠른 언어
- 구조화 프로그래밍(Structured Programming) 구현
- 시스템 프로그래밍(System Programming) 가능
- 다양한 데이터 유형(Data Type)
- 가독성(Readability)이 뛰어나 유지보수(Maintenance)
- 모듈(Module)과 함수(Function)에 의한 프로그램 구조
- Unix에서 여러 s/w 개발 가능
- 프로그래머에 대한 융통성 제공

GCC

- **GCC (GNU Compiler Collection)**

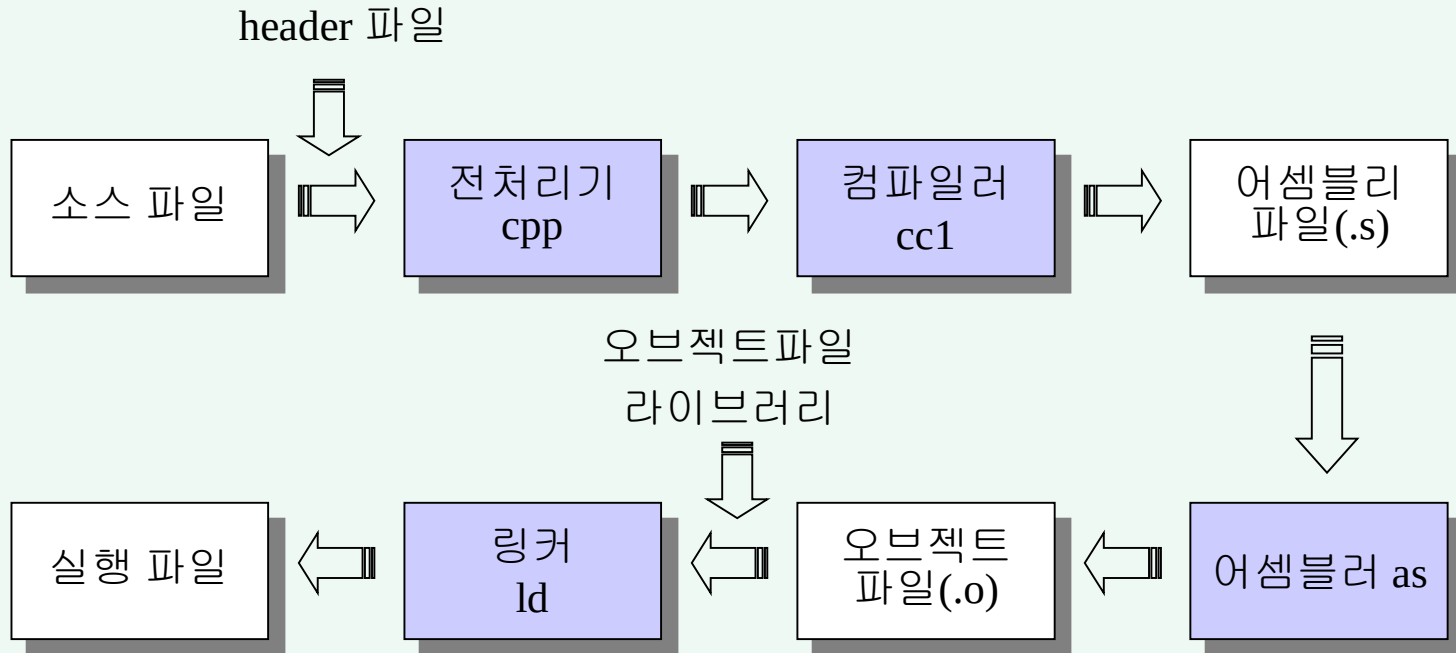
- 수 많은 컴퓨터 프로그래밍 언어들을 위한 라이브러리와 컴파일러들의 모음
- C, C++, Objective-C, Pascal, Fortran, Java, Ada 등의 언어 지원
- GCC 패키지의 주요 프로그램

파일	설 명
gcc	GNU C 와 C++ 컴파일러, cc1 과 g++ 의 기능을 모두 포함
cpp	C 언어 전처리기
cc1	실제 C 컴파일러
as	어셈블러
ld	링커

- 공식 홈페이지 : <http://www.gnu.org/software/gcc/>

GCC

- 소스 프로그램부터 최종 실행 파일이 만들어지는 과정



실행 예제

- GCC 정보

```
[cprog2@seps5 cprog2]# gcc -v
Reading specs from /usr/lib/gcc-lib/i386-hancom-linux/3.2.3/specs
Configured with: ../configure --prefix=/usr --mandir=/usr/share/man
--infodir=/usr/share/info --enable-shared --enable-threads=posix
--disable-checking --with-system-zlib --enable-__cxa_atexit --
host=i386-hancom-linux
Thread model: posix
gcc version 3.2.3 20030422 (Hancom Linux 3.2.3)
[cprog2@seps5 cprog2]#
```

GCC

- 실행 방법

- “gcc” 명령 실행

- 컴파일과 링크를 동시에 수행

- 간단한 실행 예제

- C 언어 소스 파일(.c) 작성 후 실행하면, “a.out” 실행파일로 만들

```
#include <stdio.h>
main()
{
    printf("Hello, World\n");
}
```

```
[cprog2@seps5 cprog2]# gcc hello.c
[cprog2@seps5 cprog2]# ./a.out
Hello, World
[cprog2@seps5 cprog2]#
```

GCC

- gcc 명령어 옵션

옵션	기능
-v	실행 명령어들과 버전을 출력한다
-E	전처리만 실행한다; 컴파일하거나 어셈블하지 않는다
-S	컴파일만 실행한다; 어셈블하거나 링크하지 않는다
-c	컴파일 또는 어셈블한다; 링크하지 않는다
-g	운영체제 고유 형식으로 디버깅 정보를 만든다
-o <file>	출력을 file 에 둔다
-I<dir>	헤더 파일을 검색할 디렉토리를 추가한다
-L<dir>	-l 을 위해 검색할 디렉토리를 추가한다
-D<macro>	매크로를 미리 지정한다
-O<level>	최적화 수준을 지정한다. level 이 없으면 -O1 과 같다
-l<lib>	링크할 라이브러리 파일을 지정한다

GCC 실행 예제

- 출력 파일 지정 예

```
[cprog2@seps5 cprog2]# gcc -o hello hello.c  
[cprog2@seps5 cprog2]# ./hello  
Hello, World  
cprog2@seps5 cprog2]#
```


GCC 실행 예제

- 전처리 예

- 전처리를 수행하면, C 구문에서 “#”으로 시작하는 모든 코드가 해석되어 소스에 포함된다.
 - 헤더 파일들, 매크로 등

```
[cprog2@seps5 cprog2]# gcc -E -o hello.i hello.c
[cprog2@seps5 cprog2]# cat hello.i
# 1 "hello.c"
# 1 "<built-in>"
# 1 "<command line>"
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3
....
[cprog2@seps5 cprog2]#
```

GCC 실행 예제

- 어셈블러
 - 어셈블리 코드 생성
 - CPU 종류마다 어셈블리 명령어가 다름

```
[cprog2@seps5 cprog2]# gcc -S hello.c
[cprog2@seps5 cprog2]# cat hello.s
.file "hello.c"
.section .rodata
.LC0:
.string "Hello, WorldWn"
.text
.globl main
.type main,@function
main:
    pushl   %ebp
    movl    %esp, %ebp
    subl    $8, %esp
    andl    $-16, %esp
    movl    $0, %eax
    subl    %eax, %esp
    subl    $12, %esp
    pushl    $.LC0
    call    printf
    addl    $16, %esp
    leave
    ret
.Lfe1:
    .size   main,.Lfe1-main
    .ident  "GCC: (GNU) 3.2.3 20030422
(Hancom Linux 3.2.3)"
[cprog2@seps5 cprog2]#
```

GCC

- 프로그램 링킹(Linking)

- 하나 또는 여러 개의 목적(Object) 파일을 통합하여 하나의 실행 가능한 실행 프로그램으로 만드는 작업
- 링커(Linker) – 링킹에 사용하는 도구 프로그램
 - gcc, ld
- 로더(Loader) – 실행 프로그램을 주기억장치에 설치한 후 실행시킴

- 링킹 예

```
[cprog2@seps5 cprog2]# gcc -c hello.c
[cprog2@seps5 cprog2]# ls
hello.c  hello.o
[cprog2@seps5 cprog2]#
```

```
[cprog2@seps5 cprog2]# gcc -o hello hello.o
[cprog2@seps5 cprog2]# ls
hello  hello.c  hello.o
[cprog2@seps5 cprog2]#
```

GCC 실행 예제

- 소스 파일이 2개 이상인 경우의 링킹 예

```
/* main.c */
#include <stdio.h>
extern void sub();
main()
{
    printf("This is main file.\n");
    sub();
}
```

```
/* sub.c */
sub()
{
    printf("This is sub file.\n");
}
```

```
[cprog2@seps5 cprog2]# gcc -c main.c
[cprog2@seps5 cprog2]# gcc -c sub.c
[cprog2@seps5 cprog2]# gcc -o main main.o sub.o
[cprog2@seps5 cprog2]# ./main
This is main file.
This is sub file.
[cprog2@seps5 cprog2]#
```

GCC 실행 예제

- 헤더 파일 포함(include)
 - 헤더파일이 “header” 라는 서브디렉토리에 있을 때

```
/* main.c */
#include <stdio.h>
#include "main.h"
main()
{
    printf("This is %s file.\n",
MAIN);
}
```

```
/* main.h */
#define MAIN "main.c"
```

```
[cprog2@seps5 cprog2]# gcc -o main -I./header main.c
[cprog2@seps5 cprog2]# ./main
This is main.c file.
[cprog2@seps5 cprog2]#
```

라이브러리

* 링크의 종류

– 정적 링크(static linking)

- 최종 실행 파일에 필요한 오브젝트 파일들을 미리 링크하여 실행 파일에 함께 포함
- 장점 - 실행 파일만 있으면 별도의 파일 없이 실행 가능
- 단점 - 실행파일 크기가 커지고, 라이브러리 갱신 시 관련된 실행 파일들을 모두 컴파일하여 갱신

– 동적 링크(dynamic linking)

- 필요한 파일들을 미리 링크하지 않고, 실행하려고 할 때 필요한 프로그램 모듈들을 결합하여 실행 계속
- 장점 - 실행파일 크기가 작아져 메모리를 조금 차지
- 단점 - 실행 파일 이외에 별도의 필요한 라이브러리를 제공
- UNIX 또는 LINUX 에서 대부분의 실행 파일에 해당

라이브러리

- 유용한 기능을 가진 프로그램 모듈들을 모아 놓은 파일
 - C library(libc) – printf 등의 C 표준 함수들 제공
 - Math library(libm) – sin 등의 수학 함수들 제공
 - UNIX/LINUX 에서 /lib, /usr/lib, /usr/local/lib 등에 둬
- 라이브러리의 종류
 - 정적 라이브러리(static library) – 정적 링킹 시 사용
 - .a 확장자를 가짐
 - 공유 라이브러리(shared library) – 동적 링킹 시 사용
 - .so 확장자를 가짐
 - 동적 라이브러리(dynamic library) – 동적 로딩 시 사용
 - 실행 도중에 동적으로 로딩됨
 - 필요한 라이브러리가 수시로 등록, 실행, 제거 가능하므로 유연함
 - 예) 웹 응용에서 플러그인 모듈

라이브러리

- 라이브러리 관련 gcc 옵션

옵션	기능
-L경로	라이브러리가 있는 경로를 삽입
-l라이브러리	라이브러리가 있는 표준 디렉토리 "/usr/lib", "/lib"를 탐색

- 링킹 예제

```
1  /* ex3_5.c */
2  #include <stdio.h>
3  #include <math.h>
4  #define PI    3.14159265
5
6  main()
7  {
8      printf("sin(PI/2) = %g.\n", sin(PI/2));
9  }
```


라이브러리

* 링킹 및 실행

```
[cprog2@seps5 cprog2]# gcc -o ex3_5 ex3_5.c -lm
[cprog2@seps5 cprog2]# ./ex3_5
sin(PI/2) = 1.
```

위의 예는 수학 라이브러리 “**libm.so**”를 링크하기 위해 **-lm** 옵션을 사용하였다.

라이브러리 링크 시 접두어 “**lib**”와 확장자 “**so**”를 빼고 이름 **m**만 사용해 **-lm** 으로 지정

* 링킹 확인 – “**ldd**” 명령 사용

```
[cprog2@seps5 cprog2]# ldd ex3_5
    libm.so.6 => /lib/libm.so.6 (0x40026000)
    libc.so.6 => /lib/libc.so.6 (0x40048000)
    /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
[cprog2@seps5 cprog2]#
```

- * 옛 버전에서는 위와 같이 수학 라이브러리를 직접 링크시켜 주어야 하였지만 현재는 표준 C라이브러리에 수학 라이브러리가 포함되어 링크시키지 않아도 되므로 다음과 같이 링킹 및 실행한다.

```
[dhju@localhost ex1]$ gcc -o ex3_5 ex3_5.c
[dhju@localhost ex1]$ ./ex3_5
sin(PI/2) = 1.000000
```

라이브러리

- 정적 라이브러리
 - 정적 라이브러리 제작
 - “ar” 명령을 사용하여 오브젝트 파일들을 묶음
 - 아카이브(.a) 파일 생성

ar	
일반형식	ar [options] archive files...
주요옵션	d : 아카이브로부터 오브젝트 모듈들을 제거 r : 아카이브에 오브젝트 모듈들을 삽입. 이전에 존재하는 같은 모듈이 있으면 새로운 모듈로 대체. t : 아카이브 내용을 출력 x : 아카이브로부터 오브젝트 모듈을 추출 c : 아카이브 파일을 생성 s : 아카이브에 오브젝트 파일 인덱스를 기록

라이브러리

- 정적 라이브러리 사용
 - 정적 라이브러리 제작 예

```
/* max.c */  
int max(int a, int b)  
{  
    if (a > b) return a;  
    else return b;  
}
```

```
/* min.c */  
int min(int a, int b)  
{  
    if (a < b) return a;  
    else return b;  
}
```

```
/* testlib.h */  
int max(int a, int b);  
int min(int a, int b);
```

```
[cprog2@seps5 lib]$ gcc -c max.c  
[cprog2@seps5 lib]$ gcc -c min.c  
[cprog2@seps5 lib]$ ls  
max.c max.o min.c min.o testlib.h  
[cprog2@seps5 lib]$ ar rcs libtest.a max.o min.o  
[cprog2@seps5 lib]$
```

- 위와 같이 하면 **lib** 디렉토리에 정적라이브러리 “**libtest.a**”가 만들어진다.

라이브러리

- 정적 라이브러리 사용

```
/* main.c */
#include <stdio.h>
#include "testlib.h"

main()
{
    printf("max(1,2) = %d\n", max(1,2));
    printf("min(1,2) = %d\n", min(1,2));
}
```

앞에서 만든 헤더파일 “**testlib.h**”와 라이브러리 “**libtest.a**”는 현재 디렉토리인 **cprog2** 아래에 **lib** 디렉토리에 만들어져 있다. 현재 디렉토리에서 다음과 같이 링크하여 실행한다.

```
[cprog2@seps5 cprog2]$ gcc -I./lib -L./lib main.c -ltest
[cprog2@seps5 cprog2]$ ls
a.out lib main.c
[cprog2@seps5 cprog2]$ ./a.out
max(1,2) = 2
min(1,2) = 1
[cprog2@seps5 cprog2]$
```

라이브러리

- 공유 라이브러리
 - 공유 라이브러리 제작
 - “gcc” 명령의 옵션 사용
 - “-shared” 는 공유 라이브러리를 사용한다는 옵션

```
[cprog2@seps5 lib]$ gcc -shared -o libtest.so max.o min.o
[cprog2@seps5 lib]$ ls
libtest.so
[cprog2@seps5 lib]$
```

```
[cprog2@seps5 cprog2]$ gcc -I./lib -L./lib main.c -ltest
[cprog2@seps5 cprog2]$ ./a.out
./a.out: error while loading shared libraries: libtest.so: cannot open shared
object file: No such file or directory
[cprog2@seps5 cprog2]$ ldd a.out
        libtest.so => not found
        libc.so.6 => /lib/libc.so.6 (0x40027000)
        /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
[cprog2@seps5 cprog2]$
```

위에서 **a.out**을 실행했는데 공유라이브러리를 **load** 하는데 오류가 발생하였다. 이는 환경변수에 공유라이브러리 경로를 지정하여 해결한다.

라이브러리

- 공유 라이브러리 사용
 - 공유 라이브러리를 로드(load)하기 위해서는 공유 라이브러리 경로가 환경변수에 설정되어 있어야 함.
“LD_LIBRARY_PATH” 환경변수에 공유라이브러리 경로를 설정하여야 함

```
[cprog2@seps5 cprog2]$ export LD_LIBRARY_PATH=~/.lib
[cprog2@seps5 cprog2]$ ldd ./a.out
    libtest.so => /home/cprog2/lib/libtest.so.1 (0x40011000)
    libc.so.6 => /lib/libc.so.6 (0x40029000)
    /lib/ld-linux.so.2 => /lib/ld-linux.so.2 (0x40000000)
[cprog2@seps5 cprog2]$ ./a.out
max(1,2) = 2
min(1,2) = 1
[cprog2@seps5 cprog2]$
```

위와 같이 “LD_LIBRARY_PATH” 환경변수에 공유라이브러리가 있는 경로인 “~/.lib” 또는 “./lib” 를 지정하면 공유라이브러리가 연결되어 load할 수 있다. 이를 ldd 명령으로 확인할 수 있다.

(참고: “~” 기호는 홈디렉토리를 의미한다.)

라이브러리

- 동적 라이브러리 사용

	동적 라이브러리 인터페이스
형식	<pre>#include <dlfcn.h> void *dlopen (const char *filename, int flag); const char *dlerror(void); void *dlsym(void *handle, char *symbol); int dlclose (void *handle);</pre>
기능	dlopen 함수는 filename 의 라이브러리를 적재한다. dlerror 함수는 dlopen, dlsym, dlclose 함수에 의해 발생한 가장 최근의 오류를 출력한다. dlsym 함수는 dlopen 에 의해 적재된 라이브러리를 사용할 수 있도록 심볼들을 찾는다. dlclose 함수는 적재된 라이브러리의 handle 에 대한 참조계수를 감소시킨다.
반환값	dlopen 함수는 성공적으로 호출되면 적재 라이브러리에 대한 handle 을 반환. 실패한 경우 NULL 을 반환. dlerror 함수는 초기에 호출되면 오류 메시지 주소 반환. 연속적으로 두 번 이상 호출되면 NULL 반환. dlsym 함수는 성공적으로 호출되면 심볼의 적재된 주소를 반환. 심볼을 찾지 못하면 NULL 반환. dlclose 함수는 성공적으로 호출되면 NULL 반환. 그렇지 않으면 다른 값 반환.

라이브러리

- 동적 라이브러리 사용

- 동적 라이브러리 인터페이스 함수들을 사용하여 소스 제작
- 링크 시 **libdl.so** 과 링크하고(-ldl), “-rdynamic” 옵션 사용
- -l 옵션으로 라이브러리 **libdl.so** 링크시 **lib** 와 **so**를 제외하고 **-ldl** 로 옵션 지정

```
/* ex3_6.c */
#include <stdio.h>
#include <dlfcn.h>

main()
{
    void *handle;
    int (*max)(int, int), (*min)(int, int);
    char *error;

    handle = dlopen (".lib/libtest.so",
RTLD_LAZY);
    if (!handle) {
        fputs (dlerror(), stderr);
        exit(1);
    }
    max = dlsym(handle, "max");
    if ((error = dlerror()) != NULL) {
        fprintf (stderr, "%s", error);
        exit(1);
    }
}
```

```
min = dlsym(handle, "min");
if ((error = dlerror()) != NULL) {
    fprintf (stderr, "%s", error);
    exit(1);
}
printf ("max(1,2)=%d\n", (*max)(1,2));
printf ("min(1,2)=%d\n", (*min)(1,2));
dlclose(handle);
}
```

◆ 실행 결과

```
[cprog2@seps5 cprog2]$ gcc -rdynamic
ex3_6.c -ldl
[cprog2@seps5 cprog2]$ ./a.out
max(1,2)=2
min(1,2)=1
[cprog2@seps5 cprog2]$
```


make 도구

* make

- 많은 프로그램 모듈로 구성된 대규모 프로그램 소스를 효율적으로 유지하고 일관성 있게 관리하도록 도와주는 도구
- 여러 파일로 구성된 파일들을 의존 관계에 따라 자동적으로 컴파일하고 링크해 주는 자동화 도구
- 여러 파일들 간의 의존성을 저장하고 수정된 파일에 연관된 소스 파일들만 재컴파일 가능

make

- **Makefile**

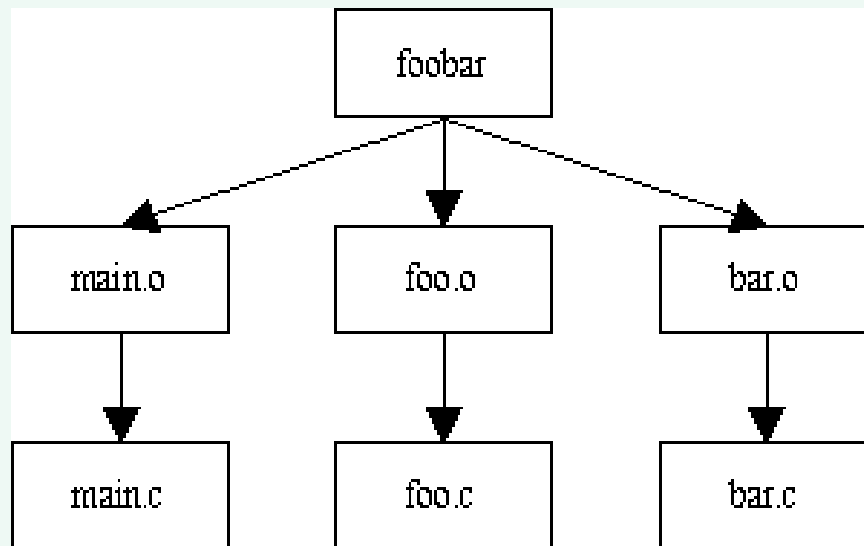
- **make** 도구에서 가장 중요한 파일
- 자동 컴파일을 위한 일종의 명령어 파일
- 최종 응용 프로그램 구성에 있어서 소스 파일들 사이의 다양한 의존성에 대한 규칙을 기술
- 형식

```
대상(TARGET) ... : 의존하는 파일들(PREREQUISITES) ...  
    명령(COMMAND)  
    ...  
    ...
```

- 대상 : 프로그램이 생성하는 목적 파일 이름 (오브젝트/실행 파일 등)
- 의존 파일 : 대상을 생성하기 위해 필요한 파일들
- 명령 : ‘make’ 가 실행하는 명령어

make

- 실행 예제
 - 최종 실행 파일 “**foobar**” 는 **main.o**, **foo.o**, **bar.o** 에 의존하며, 마찬가지로 **main.o**, **foo.o**, **bar.o** 는 각각 **main.c**, **foo.c**, **bar.c** 에 의존



make

- 실행 예제

- 의존관계

```
foobar : main.o foo.o bar.o
main.o : main.c
foo.o : foo.c
bar.o : bar.c
```

- ◆ 실행 명령

```
gcc -o foobar main.o foo.o bar.o
gcc -c main.c
gcc -c foo.c
gcc -c bar.c
```

- 최종 Makefile

```
foobar : main.o foo.o bar.o
    gcc -o foobar main.o foo.o bar.o
main.o : main.c
    gcc -c main.c
foo.o : foo.c
    gcc -c foo.c
bar.o : bar.c
    gcc -c bar.c
clean :
    rm -f foobar main.o foo.o bar.o
```

make

- 실행 예제
 - 실행 결과

```
[cprog2@seps5 make]$ make
gcc -c main.c
gcc -c foo.c
gcc -c bar.c
gcc -o foobar main.o foo.o bar.o
[cprog2@seps5 make]$
```

```
[cprog2@seps5 make]$ ./foobar
Hello, world!
Goodbye, my love.
[cprog2@seps5 make]$
```

make

- 실행 예제

- 실행 결과

- **make** 실행 후 소스 파일에 수정이 없다면 **make**를 다시 실행해도 아무 변화가 없음

```
[cprog2@seps5 make]$ make  
make: `foobar'는 이미 갱신되었습니다.  
[cprog2@seps5 make]$
```

- 일부 파일을 수정하면 수정한 파일에 관련된 부분만 재실행됨

```
[cprog2@seps5 make]$ touch main.c  
[cprog2@seps5 make]$ make  
gcc -c main.c  
gcc -o foobar main.o foo.o bar.o  
[cprog2@seps5 make]$
```

make

- clean 기능

- **make** 실행 전 상태로 되돌리기 위해, 디렉토리 파일들 정리
- 컴파일 과정에서 만들어진 오브젝트/실행 파일들을 제거
- 실행 예

```
[cprog2@seps5 make]$ make clean
rm -f foobar main.o foo.o bar.o
[cprog2@seps5 make]$ ls
Makefile bar.c foo.c main.c
[cprog2@seps5 make]$
```

[실습예제1] 다음은 4개의 파일로 구성된 C프로그램이다. 컴파일하여 실행하시오.

“example.h” →

```
#define YEAR 2020
```

“search.c” →

```
#include <stdio.h>
search()
{
    printf("This is a search function \n");
}
```

“update.c” →

```
#include <stdio.h>
update()
{
    printf("This is a update function \n");
}
```

“main.c” →

```
#include <stdio.h>
#include "example.h"
main()
{
    int year=YEAR;
    printf("This yesar is %d\n", year);
    search();
    update();
}
```


[실습예제1] 다음은 4개의 파일로 구성된 C프로그램이다. 컴파일하여 실행하시오.

이 문제를 3가지 방법으로 컴파일 및 실행하여 보자.

컴파일 및 실행방법1) 다음과 같이 한번에 컴파일하여 실행한다.

```
[dhju@localhost ex2]$ gcc main.c search.c update.c
[dhju@localhost ex2]$ ./a.out
This year is 2020
This is a search fnction
This is a update fnction
[dhju@localhost ex2]$
```

컴파일 및 실행방법2) 각각의 파일을 컴파일하여 링크하여 실행한다.

```
[dhju@localhost ex2]$ gcc -c search.c
[dhju@localhost ex2]$ gcc -c update.c
[dhju@localhost ex2]$ gcc -c main.c
[dhju@localhost ex2]$ gcc main.o search.o update.o
[dhju@localhost ex2]$ ./a.out
This year is 2020
This is a search fnction
This is a update fnction
[dhju@localhost ex2]$ _
```

컴파일 및 실행방법3) make도구를 사용하여 자동 컴파일하여 실행한다.

- 먼저 Makefile을 다음과 같이 작성한다.

```
[dhju@localhost ex2]$ cat Makefile
main : main.o search.o update.o
    gcc -o main main.o search.o update.o
main.o : main.c example.h
    gcc -c main.c
search.o : search.c
    gcc -c search.c
update.o : update.c
    gcc -c update.c

[dhju@localhost ex2]$
```

- make 명령을 실행한다.

```
[dhju@localhost ex2]$ make
gcc -c main.c
gcc -c search.c
gcc -c update.c
gcc -o main main.o search.o update.o
```

- 생성된 실행파일 main을 실행한다.

```
[dhju@localhost ex2]$ ./main
This year is 2020
This is a search fnnction
This is a update fnnction
```

make

- 주석과 매크로

- 주석 – ‘#’ 기호로부터 시작하여 줄의 끝까지 모든 문자

```
# COMMENT .....
```

- 매크로

- 긴 문자열을 하나의 매크로로 정의하여 여러 곳에 사용 가능
- 형식

```
NAME = value
```

- 정의 예

```
INCLUDES = /usr/local/include1 /usr/local/include2  
DEBUGS = -g -O2
```

make

- 매크로 사용 예

```
OBJECTS = main.o foo.o bar.o
foobar : $(OBJECTS)
    gcc -o foobar $(OBJECTS)
main.o : main.c
    gcc -c main.c
foo.o : foo.c
    gcc -c foo.c
bar.o : bar.c
    gcc -c bar.c
clean :
    rm -f foobar $(OBJECTS)
```

[실습예제2] 앞의 [실습예제1]에서 프로그램을 일부 수정해 보자. **update** 함수 일부를 다음과 같이 수정하였다. 이제 어떻게 컴파일하여 실행해야 하는가?

“**update.c**” 수정 →

```
#include <stdio.h>
update()
{
    printf("Update Function is here  \Wn");
}
```

이 경우 우리는 다시 컴파일하여 실행해야 하는데 **make**도구를 사용하여 다음과 같이 간단히 해결한다.

```
[dhju@localhost ex21]$ make
gcc -c update.c
gcc -o main main.o search.o update.o
[dhju@localhost ex21]$ ./main
This year is 2020
This is a search fmnction
Update Function is here
[dhju@localhost ex21]$
```

make 도구는 “**update.c**”파일이 수정된 것을 파악하여 “**update.c**”파일만 다시 컴파일하여 “**main**” 실행 파일을 다시 생성한다. 이와 같이 작성된 프로그램이 수정될 때마다 개발자가 매번 컴파일과 링크 작업을 반복할 필요없이 **make**도구를 사용하면 자동으로 수정된 파일을 찾아 컴파일과 링크를 해 주므로 프로그램 개발 시 아주 편리한 도구이다.